

Devoir surveillé terminal

Durée 2h00
(Corrections)

Exercice 1 (Questions de cours - 2 pts)

1°) Quelles sont les différences entre une analyse ascendante et descendante ? Donnez un exemple.

Solution : (1 pt) Le principe de l'analyse descendante consiste à partir de l'axiome, puis à effectuer des dérivations jusqu'à trouver le mot à analyser (l'arbre d'analyse est construit de la racine aux feuilles). L'analyse ascendante part du mot et tente de remonter jusqu'à l'axiome (l'arbre d'analyse est construit des feuilles jusqu'à la racine).

Soit la grammaire définie par $S \rightarrow aSb \mid ab$ et le mot à analyser $aabb$:

- Analyse descendante : $S \vdash aSb \vdash aabb$
- Analyse ascendante : $aabb \vdash aSb \vdash S$

2°) Lors de la génération de quadruplets, il est parfois nécessaire de faire des retours en arrière pour modifier des quadruplets. Expliquez pourquoi et donnez un exemple.

Solution : (1 pt) La génération des quadruplets est réalisée au fur-et-à-mesure de l'analyse. Les branchements (sauts) ont besoin de connaître les numéros des quadruplets de destination qui ne sont pas encore connus au moment de l'analyse. Par exemple avec la conditionnelle : les quadruplets du test sont générés et en fonction s'il est vrai ou faux, un saut doit être effectué soit dans la première partie ou soit dans la deuxième. L'adresse de la deuxième partie n'est connue qu'une fois les quadruplets de la première partie générés. La première partie est également terminée par un saut qui a besoin de connaître le numéro du quadruplet suivant la conditionnelle.

Exercice 2 (Analyseur LL(1) - 8 pts)

Soit la grammaire G suivante :

$$\begin{aligned} F &\rightarrow T \text{ id } (L) ; & (1) \\ L &\rightarrow \epsilon & (2) \\ L &\rightarrow T \text{ id } L' & (3) \\ L' &\rightarrow , T \text{ id } L' & (4) \\ L' &\rightarrow \epsilon & (5) \\ T &\rightarrow \text{int} & (6) \\ T &\rightarrow \text{double} & (7) \end{aligned}$$

1°) Donnez la définition formelle de cette grammaire.

Solution : (1 pt) Soit la grammaire $G = (T, N, R, S)$ avec :

- $T = \{\text{id}, '(', ')', ',', ';', 'int', 'double'\}$
- $N = \{F, L, L', T\}$
- $S = F$
- $R = \{F \rightarrow T \text{ id } (L) ; , L \rightarrow \epsilon , L \rightarrow T \text{ id } L' , L' \rightarrow , T \text{ id } L' , L' \rightarrow \epsilon , T \rightarrow \text{int} , T \rightarrow \text{double}\}$

2°) Construisez les ensembles *Premier* et *Suivant* des symboles non terminaux.

Solution : (2 pts)

- $Premier(T) = \{ int, double \}$
- $Premier(L) = \{ \epsilon, int, double \}$
- $Premier(L') = \{ ', ', \epsilon \}$
- $Premier(F) = \{ int, double \}$
- $Suivant(T) = \{ id \}$
- $Suivant(L) = \{ ') ' \}$
- $Suivant(L') = \{ ') ' \}$
- $Suivant(F) = \{ \epsilon \}$

3°) Cette grammaire est-elle LL(1) ?

Solution : (1 pt)

- Pour T : $Premier(int) = int$ et $Premier(double) = double$
- Pour L : $Premier(T id L') = int, double$ et $Premier(\epsilon) = \epsilon$
- On a $L \rightarrow \epsilon$ et $L' \rightarrow \epsilon$ mais les autres parties de droite ne dérivent sur ϵ

La grammaire est donc LL(1).

Une autre solution : si on arrive à construire la TASD, correspondant à la question 4), alors la grammaire est LL(1).

4°) Construisez la TASD.

Solution : (2 pts) Voici la table obtenue :

	(	)	;	,	id	int	double	⊥
F						$F \rightarrow T id (L) ;$	$F \rightarrow T id (L) ;$	
L		AVC				$L \rightarrow T id L'$	$L \rightarrow T id L'$	
L'		AVC		$L' \rightarrow, T id L'$				
T						$T \rightarrow int$	$T \rightarrow double$	
(	AVCTS							
)		AVCTS						
;			AVCTS					
,				AVCTS				
id					AVCTS			
int						AVCTS		
double							AVCTS	
⊥								ACC

5°) Exécutez l'analyseur sur le mot suivant (précisez la cible, le texte source et les actions) :

```
int toto(int a, double b);
```

Solution : (2 pts)

<i>Cible</i>	<i>Texte source</i>	<i>Action</i>
$F \dashv$	$int\ toto(int\ a,\ double\ b);\ \dashv$	$F \rightarrow T\ id\ (L);\$
$T\ id\ (L);\ \dashv$	$int\ toto(int\ a,\ double\ b);\ \dashv$	$T \rightarrow int$
$int\ id\ (L);\ \dashv$	$int\ toto(int\ a,\ double\ b);\ \dashv$	$AVCTS$
$id\ (L);\ \dashv$	$toto(int\ a,\ double\ b);\ \dashv$	$AVCTS$
$(L);\ \dashv$	$(int\ a,\ double\ b);\ \dashv$	$AVCTS$
$L);\ \dashv$	$int\ a,\ double\ b);\ \dashv$	$L \rightarrow T\ id\ L'$
$T\ id\ L');\ \dashv$	$int\ a,\ double\ b);\ \dashv$	$T \rightarrow int$
$int\ id\ L');\ \dashv$	$int\ a,\ double\ b);\ \dashv$	$AVCTS$
$id\ L');\ \dashv$	$a,\ double\ b);\ \dashv$	$AVCTS$
$L');\ \dashv$	$,\ double\ b);\ \dashv$	$L' \rightarrow,\ T\ id\ L'$
$,\ T\ id\ L');\ \dashv$	$,\ double\ b);\ \dashv$	$AVCTS$
$T\ id\ L');\ \dashv$	$double\ b);\ \dashv$	$T \rightarrow double$
$double\ id\ L');\ \dashv$	$double\ b);\ \dashv$	$AVCTS$
$id\ L');\ \dashv$	$b);\ \dashv$	$AVCTS$
$L');\ \dashv$	$);\ \dashv$	AVC
$);\ \dashv$	$);\ \dashv$	$AVCTS$
$;\ \dashv$	$;\ \dashv$	$AVCTS$
$\dashv$	$\dashv$	ACC

Exercice 3 (Analyseur LR(1) - 6 pts)

Soit la grammaire suivante :

$$\begin{aligned}
 G &\rightarrow R & (1) \\
 G &\rightarrow R ; G & (2) \\
 R &\rightarrow id : L & (3) \\
 L &\rightarrow id L & (4) \\
 L &\rightarrow \epsilon & (5)
 \end{aligned}$$

1°) Construisez les ensembles *Premier* et *Suivant* des symboles non terminaux.

Solution : (1 pt)

- $Premier(G) = \{ id \}$
- $Premier(R) = \{ id \}$
- $Premier(L) = \{ id, \epsilon \}$
- $Suivant(G) = \{ \epsilon \}$
- $Suivant(R) = \{ \epsilon, ; \}$
- $Suivant(L) = \{ \epsilon, ; \}$

2°) Construisez les ensembles d'items LR(1).

Solution : (3 pt) On ajoute la règle $G' \rightarrow G \dashv$

- $I_0 = Fermeture([G' \rightarrow .G \dashv, \dashv]) = \{ [G' \rightarrow .G \dashv, \dashv], [G \rightarrow .R, \dashv], [G \rightarrow .R; G, \dashv], [R \rightarrow .id : L, \{ \dashv, ; \}] \}$
- $I_1 = Transition(I_0, G) = \{ [G' \rightarrow G. \dashv, \dashv] \}$
- $I_2 = Transition(I_0, R) = \{ [G \rightarrow R., \dashv], [G \rightarrow R.; G, \dashv], \}$
- $I_3 = Transition(I_0, id) = \{ [R \rightarrow id. : L, \{ \dashv, ; \}] \}$
- $I_4 = Transition(I_2, ;) = \{ [G \rightarrow R; .G, \dashv], [G \rightarrow .R, \dashv], [G \rightarrow .R; G, \dashv], [R \rightarrow .id : L, \{ \dashv, ; \}] \}$
- $I_5 = Transition(I_3, :) = \{ [R \rightarrow id : .L, \{ \dashv, ; \}], [L \rightarrow .idL, \{ \dashv, ; \}], [L \rightarrow .\epsilon, \{ \dashv, ; \}] \}$
- $I_6 = Transition(I_4, G) = \{ [G \rightarrow R; G., \dashv] \}$
- $Transition(I_4, R) = I_2$
- $Transition(I_4, id) = I_3$

- $I_7 = \text{Transition}(I_5, L) = \{ [R \rightarrow id : L, \{ \neg, ; \}], \}$
- $I_8 = \text{Transition}(I_5, id) = \{ [L \rightarrow id.L, \{ \neg, ; \}], [L \rightarrow .idL, \{ \neg, ; \}], [L \rightarrow .\epsilon, \{ \neg, ; \}] \}$
- $I_9 = \text{Transition}(I_8, L) = \{ [L \rightarrow idL., \{ \neg, ; \}], \}$
- $\text{Transition}(I_8, id) = I_8$

3°) Construisez la table d'analyse selon la méthode LR(1).

Solution : (2 pt) Voici la table :

États	Action				Successeur		
	id	;	:	$\neg$	G	R	L
0	d,3				1	2	
1				accepter			
2		d,4		r,1			
3			d,5				
4	d,3				6	2	
5	d,8	r,5		r,5			7
6				r,2			
7		r,3		r,3			8
8	d,8	r,5		r,5			9
9		r,4		r,4			

4°) [bonus] Est-il possible de construire un analyseur LALR(1) ?

Solution : (1 pt) Les cœurs des ensembles d'items sont différents. Donc ce n'est pas possible.

Exercice 4 (Décoration de grammaire - 4 pts)

Nous souhaitons pouvoir créer des arbres d'évaluation typés (des arbres syntaxiques représentant une expression arithmétique) à partir de la grammaire suivante :

$$\begin{aligned}
 E &\rightarrow E_1 + T \mid T \\
 T &\rightarrow T_1 * F \mid F \\
 F &\rightarrow id \mid entier \mid reel \mid (E)
 \end{aligned}$$

Nous considérons les fonctions suivantes (la table des symboles est considérée existante et accessible à tout endroit ; elle constituée d'éléments de type `entree_t`) :

```

/**
 * Création d'une feuille.
 * @param type ID ou CST
 * @param valeur un entier, un réel ou un entree_t
 * @return un noeud
 */
creer_feuille(type, valeur) : noeud_t

/**
 * Création d'un noeud.
 * @param type opération ADD ou MUL ou conversion CONV_INT ou CONV_DOUBLE
 * @param noeud1 le premier argument
 * @param noeud2 le deuxième argument (NULL pour CONV_INT et CONV_DOUBLE)
 * @return un noeud
 */
creer_noeud(type, noeud1, noeud2) : noeud_t

/**
 * Retourne le type d'une entrée de la table des symboles.
 * @param str le nom de l'entrée
 * @return le type de l'entrée (constante INT ou DOUBLE)
 */

```

```

type_entree(str) : int

/**
 * Retourne l'entrée de la table des symboles.
 * @param str le nom de l'entrée
 * @return l'entrée
 */
entree(str) : entree_t

```

1°) Quels sont les attributs nécessaires pour chaque symbole de la grammaire ?

Solution : (1 pt) Nous pouvons spécifier un attribut `lex` permettant de récupérer la valeur envoyée par l'analyseur lexical (pour `id`, entier et reel). Chaque symbole non terminal E , T , F possède les attributs `arbre` et `type`.

2°) Proposez une décoration de la grammaire permettant de créer les arbres d'évaluation.

Solution : (3 pts)

```

E -> E_1 + T {
    Si E_1.type = T.type Alors
        E.arbre = creer_noeud(ADD, E_1.arbre, T.arbre)
        E.type = E_1.type
    Sinon
        Si E_1.type = INT Alors
            tmp = creer_noeud(CONV_REEL, E_1.arbre, NULL)
            E.arbre = creer_noeud(ADD, tmp, T.arbre)
        Sinon
            tmp = creer_noeud(CONV_REEL, T.arbre, NULL)
            E.arbre = creer_noeud(ADD, E_1.arbre, tmp)
        FinSi
        E.type = DOUBLE
    FinSi
}
E -> T {
    E.arbre = T.arbre
    E.type = T.type
}
T -> T_1 * F {
    Si T_1.type = F.type Alors
        T.arbre = creer_noeud(MUL, T_1.arbre, F.arbre)
        T.type = T_1.type
    Sinon
        Si T_1.type = INT Alors
            tmp = creer_noeud(CONV_REEL, T_1.arbre, NULL)
            T.arbre = creer_noeud(MUL, tmp, F.arbre)
        Sinon
            tmp = creer_noeud(CONV_REEL, T_1.arbre, NULL)
            T.arbre = creer_noeud(MUL, T_1.arbre, tmp, F.arbre)
        FinSi
        T.type = DOUBLE
    FinSi
}
F -> id {
    F.arbre = creer_feuille(ID, entree(id.lex))
    F.type = type_entree(id.lex)
}
F -> entier {
    F.arbre = creer_feuille(CST, entier.lex)
    F.type = INT
}

```

```
F -> reel {  
    F.arbre = creer_feuille(CST, reel.lex)  
    F.type = REEL  
}  
F -> (E) {  
    F.arbre = E.arbre  
    F.type = E.type  
}
```